

Vererbung

1 Vererbung

Mit Vererbung kann eine Klasse die Eigenschaften und Methoden einer anderen Klasse übernehmen, ohne dass Code wiederholt werden muss.

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
class Teacher extends Person {  
    Teacher(String name, int age) {  
        super(name, age);  
    }  
}
```

`super` ruft den Konstruktor der Oberklasse auf. Dadurch werden in diesem Beispiel die Eigenschaften `name` und `age` gesetzt.

```
var herrMüller = new Teacher("Herr Müller", 50);
```

Auch Methoden werden bei der Vererbung übernommen.

```
class Person {  
    ...  
    public void greet() {  
        println("Hello, my name is " + name + " and I am " + age + " years old.");  
    }  
}
```

```

}

class Teacher extends Person {
    ...
}

var herrMüller = new Teacher("Herr Müller", 50);
herrMüller.greet();

```

Hello, my name is Herr Müller and I am 50 years old.

2 Zusätzliche Eigenschaften und Methoden

In der abgeleiteten Klasse können zusätzlich Eigenschaften und Methoden definiert werden.

```

class Teacher extends Person {

    private String classRoom;

    public Teacher(String name, int age, String classRoom) {
        super(name, age);
        this.classRoom = classRoom;
    }

    public void teach() {
        println("Ich unterrichte jetzt in Raum " + classRoom + ".");
    }
}

var herrMüller = new Teacher("Herr Müller", 50, "G402");
herrMüller.teach();

```

Ich unterrichte jetzt in Raum G402.

3 Sichtbarkeit mit protected

Der folgende Code kann nicht kompiliert werden:

```

class Teacher extends Person {
    ...
}

```

```

    public boolean isRetired() {
        return age > 65;
    }
    ...
}

```

Das Problem liegt darin, dass die Eigenschaft `age` in der Klasse `Person` als `private` deklariert ist. Deshalb kann die Klasse `Teacher` nicht auf diese Eigenschaft zugreifen.

Das lässt sich beheben, indem die Eigenschaft als `protected` deklariert wird. Dann können abgeleitete Klassen darauf zugreifen. Wichtig in Java: Auch Klassen im selben Paket (also nicht nur Unterklassen) dürfen auf `protected`-Attribute zugreifen.

```

class Person {
    protected String name;
    protected int age;
    ...
}

```

```

var herrMüller = new Teacher("Herr Müller", 50);
println(herrMüller.isRetired());

```

```

false

```

4 Überschreiben von Methoden

Beim Überschreiben werden Methoden ersetzt, die in der Oberklasse bereits implementiert sind. Im folgenden Beispiel wird die Methode `greet` der Klasse `Person` in der Klasse `Student` überschrieben.

```

class Person {
    protected String name;
    protected int age;

    ...

    void greet() {
        println("Hello, my name is " + name + " and I am " + age + " years old.");
    }
}

class Student extends Person {
    ...
}

```

```

@Override
void greet() {
    println("Hi, ich bin " + name + ".");
}
}

```

```
var pana = new Student("Pana", 17);
```

```
pana.greet();
```

```
Hi, ich bin Pana.
```

5 Typ der Variablen und tatsächlicher Objekttyp

Jedes Objekt der Klasse Teacher ist auch ein Objekt der Klasse Person.

```
Person frauMaier = new Teacher("Frau Maier", 30);
```

Wenn wir dieses Objekt in einer Variablen vom Typ Person speichern, können wir nur Methoden aufrufen, die in der Klasse Person deklariert sind.

```
frauMaier.greet();
```

```
Hello, my name is Frau Maier and I am 30 years old.
```

```
frauMaier.teach(); // geht nicht
```

6 Polymorphie

Die Klasse Person hat eine Methode greet. Deshalb können wir greet auf Variablen vom Typ Person aufrufen. Welche konkrete Implementierung ausgeführt wird, hängt vom tatsächlichen Objekttyp ab.

```

class Utils {
    static void letGreetNTimes(Person person, int times) {
        for (int i = 0; i < times; i++) {
            person.greet();
        }
    }
}

```

Erst zur Laufzeit wird entschieden, welche greet-Methode (zum Beispiel aus Student oder Teacher) ausgeführt wird.

```
var pana = new Student("Pana", 17);  
Utils.letGreetNTimes(pana, 2);
```

```
Hi, ich bin Pana.  
Hi, ich bin Pana.
```

```
var herrMüller = new Teacher("Herr Müller", 50);  
Utils.letGreetNTimes(herrMüller, 3);
```

```
Hello, my name is Herr Müller and I am 50 years old.  
Hello, my name is Herr Müller and I am 50 years old.  
Hello, my name is Herr Müller and I am 50 years old.
```

Außerdem können wir Objekte beider Klassen in einer List<Person> speichern.

```
List<Person> persons = List.of(pana, herrMüller);
```

Wir können außerdem Methoden schreiben, denen eine List<Person> übergeben wird.

```
static void letAllGreet(List<Person> persons) {  
    for (Person person : persons) {  
        person.greet();  
    }  
}
```

```
Utils.letAllGreet(persons);
```

```
Hi, ich bin Pana.  
Hello, my name is Herr Müller and I am 50 years old.
```

7 Abstrakte Klassen

Um zu verhindern, dass Objekte der Klasse Person erzeugt werden, können wir die Klasse als abstrakt deklarieren.

```
abstract class Person {
```

```
var herrMüller = new Person("Herr Müller", 50); // geht nicht mehr
```

In abstrakten Klassen können auch Methoden als abstrakt deklariert werden. Dabei gibt man nur den Methodenkopf an.

```
abstract class Person {  
    abstract void greet();  
}
```

Jetzt sind alle nicht-abstrakten Klassen, die von `Person` erben, gezwungen, die Methode `greet` zu implementieren. Deshalb können wir weiterhin auf jeder Variablen vom Typ `Person` die Methode `greet` aufrufen. Methoden wie `letGreetNTimes` funktionieren damit unverändert.

```
class Utils {  
    static void letGreetNTimes(Person person, int times) {  
        for (int i = 0; i < times; i++) {  
            person.greet();  
        }  
    }  
}
```